Stochastic processes: Vicious walkers

FYTN03

Author:

Fredrik Bergelv
fredrik.bergelv@live.se

Lund University, Department of Physics

under the direction of Victor Olariu



LUND UNIVERSITY

December 13, 2024

1 Introduction

The goal of this project was to use numerical methods to conduct simulations involving randomized events. The first task was to analyse the movement of vicious walking cows on a two-dimensional field. The cows could walk randomly in any direction, but if a cow stepped into the same position as another cow, both would die. Hence the name: vicious walking cows. The objective was to simulate these events until all cows had died and analyse the results.

The second task focused on studying the movement of these cows across a minefield. This was accomplished by having one cow at a time attempt to traverse a minefield with specific dimensions and mine density. The goal of the cow was to cross the field; however, each time it stepped on a mine, both the cow and the mine would be removed, and a new cow would have to start over until it reached the end. This time, the cow walked randomly with a bias toward moving forward, reflecting its desire to make it across the field. The number of cows it took to successfully cross was collected and analysed.

2 The Algorithm

2.1 Vicious Walking Cows

The key steps to constructing the algorithm for the vicious walking cows are as follows. To initialize the problem, the field with cows had to be created. This was done by placing a specific number of cows uniquely and randomly on a field with a specific grid size. The cows were identified with the number 1, while the other grid spaces had the value 0. The movement of the cows was managed through a loop where, at each step, a randomly selected cow was moved in one of four directions, taking into account Dirichlet boundary conditions to avoid moving outside of the grid. The Dirichlet boundary condition checked if the moving cow was near any border; if it was not, the movement in that direction was added to a list of possible directions. The movement was executed via a function that adjusted the cow's position by adding or subtracting 1, the value of the moving cow. Thus, if a cow moved to the position of another, their values were combined. This was detected in the loop, where every grid value of 2 was removed. The loop continued until all cows had been killed or a maximum number of steps had been reached. The simulation was then displayed as a video.

The video provided a good view of the simulation, allowing one to see if the simulation functioned as intended. Thus, one could verify that the cow moved one step at a time, obeyed the Dirichlet boundary conditions, and correctly eliminated other cows. The simulation was also run with a single cow to confirm that the cow then moved indefinitely. Additionally, the results were interpreted through two additional plotting functions. One involved a function that plotted the number of iterations it took for all the cows to die. This plot should exhibit a distribution with a global peak and decreasing values as one moves away from the peak. Another function showed the cow density ρ_{cow} for each step, verifying that the cow density decreased continuously. This decrease should resemble the form

$$\rho_{\text{cow}}(\text{steps}) \propto \frac{1}{\text{steps}^n}, \tag{1}$$

when steps $\longrightarrow \infty$ for a system of indefinite size [1]. This equation is derived from the diffusion equation when solved for a system with initial and boundary conditions. Solving this gives $n = 1$ for a two-dimensional system, which we should obtain by fitting this equation onto the data from the simulation. This is, therefore, a very good test to see if the result obtained is valid.

2.2 Cows on a Minefield

The algorithm for the cows walking across the minefield was implemented in a similar manner; however, it utilized two grids instead of one. The grid for the minefield was initialized as before, but this time it obeyed the condition that the first and last rows could not contain mines. This was done because the goal was to simulate a cow starting on one side and trying to cross, with a minefield in between. The grid for the cow was initialized by placing one cow in the middle of the first row. The cow had a 50% chance of moving forward and an approximately 17% chance of moving in any other direction. This uneven probability arises from the cow's desire to step forward, as it wants to reach the other side. A while loop was created for the simulation, where one cow started walking from the initialized position until it either hit a mine (resulting in the removal of both the mine and the cow) or made it to the other side, at which point the simulation was stopped.

Several methods were used to analyse whether the results were correct. Firstly, the simulation was saved as a video for analysis. The video made it possible to verify that the cow moved correctly, that both the cow and the mine disappeared if the cow stepped on one, and that the simulation repeated itself when necessary and ended when it should. Secondly, a graph was created similarly to before, where the final value of the number of cows it took to get across, $N_{\text{dead cows}}$, was compared across several runs of the simulation. This should yield a distribution with a global peak and decreasing values as one moves away from the peak. If the distribution was normally Gaussian distributed, a Gaussian was fitted to the data. This was done for a specific number of mines per area, ρ_{mine}. Lastly, the mean value of the number of iterations it took for the cows to cross was compared to the mine density for a certain field depth. This comparison aimed to investigate the relationship between these two parameters, as a correlation between them is expected.

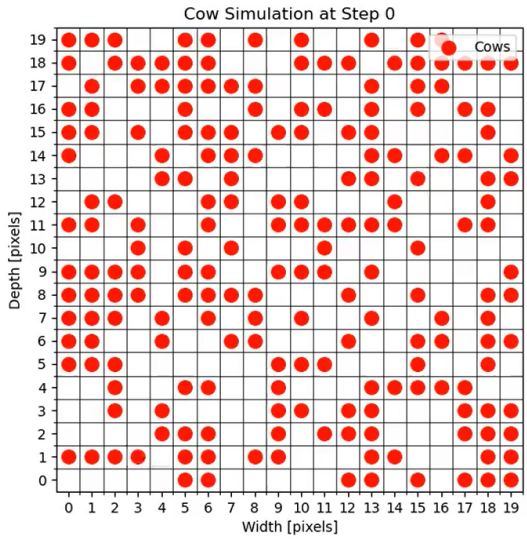
3 Result and Analysis

3.1 Vicious Walking Cows

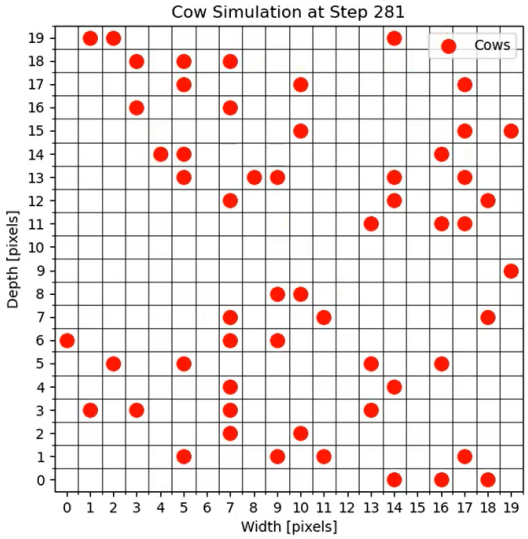
Seeing the snapshots in Figure 1, one can observe that the vicious walking cows indeed disperse over time. From the video, it is clear that the cows obey the rules stipulated above. An interesting result is that the density of the cows, ρ_{cows}, seems to decrease more quickly in the beginning and more slowly toward the end. The density of the cows, ρ_{cows}, can be seen in Figure 2. There, it is clear that the cows decrease more rapidly in the beginning, and the decrease slows down toward the end.

In Figure 3, one can see that the distribution of the iterations until $\rho_{\text{cow}} = 0$ varies as predicted. There is a distribution with a global peak and decreasing values as one moves away from the peak; however, this is not a normally distributed Gaussian. Instead, this shape resembles a Gaussian with an exponential tail. The reason for this tail is likely that the chance for the last two cows to meet may require many iterations. This reflects the randomness of the system. However, since the system is inherently crowded, it is more likely to converge around the mean convergence time rather than take a very long time. Thus, it is reasonable that we observe a Gaussian-like shape with an exponential tail.

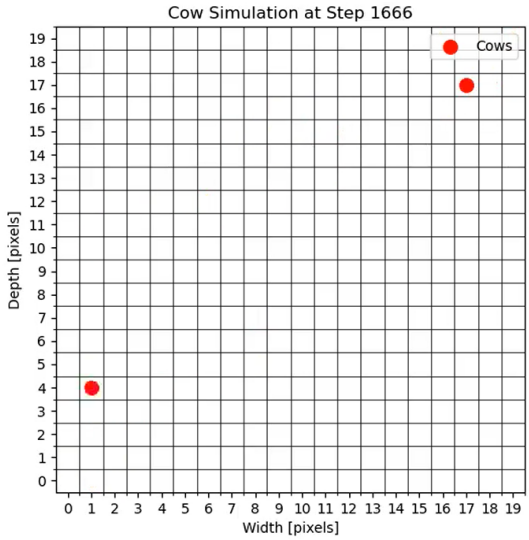
In Figure 4a, one can see that the decrease of the cows fits the theoretical Equation 1, with an exponent of 2.18. The reason why the exponent is not closer to 1 is probably due to the small size of the system. This system experiences a lot of Dirichlet boundary conditions that an ideal system would not have. In Figure 4b, one can, on the other hand, see a system with an area four times larger. This system, with an area of 160 000pixels^2, yielded an exponent of $n = 0.99$, which is very close to the theoretical value of $n = 1$.



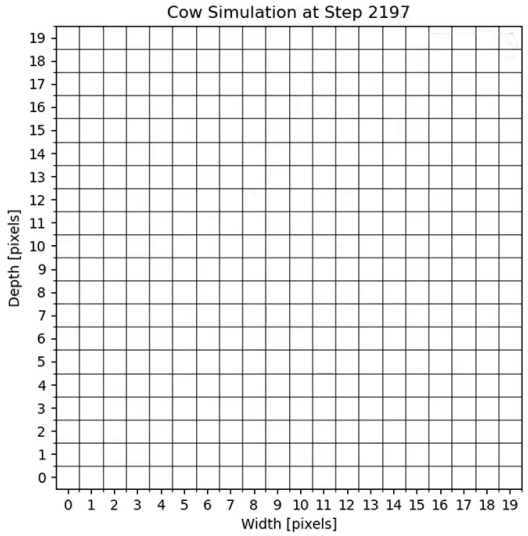
(a) The initial position of the cows.



(b) The cows left after a few iterations.



(c) Only two cows left.



(d) After 2197 iterations, all cows are dead.

Figure 1: These images from the video show the progress of the vicious walking cows from their initial positions until all have died at iteration 2197. This video started with 200 cows on a grid with dimensions 200 pixels $\times$ 200 pixels and thus an area of 400 pixels^2, resulting in an initial cow density of $\rho^0_{\text{cow}} = 0.50$.

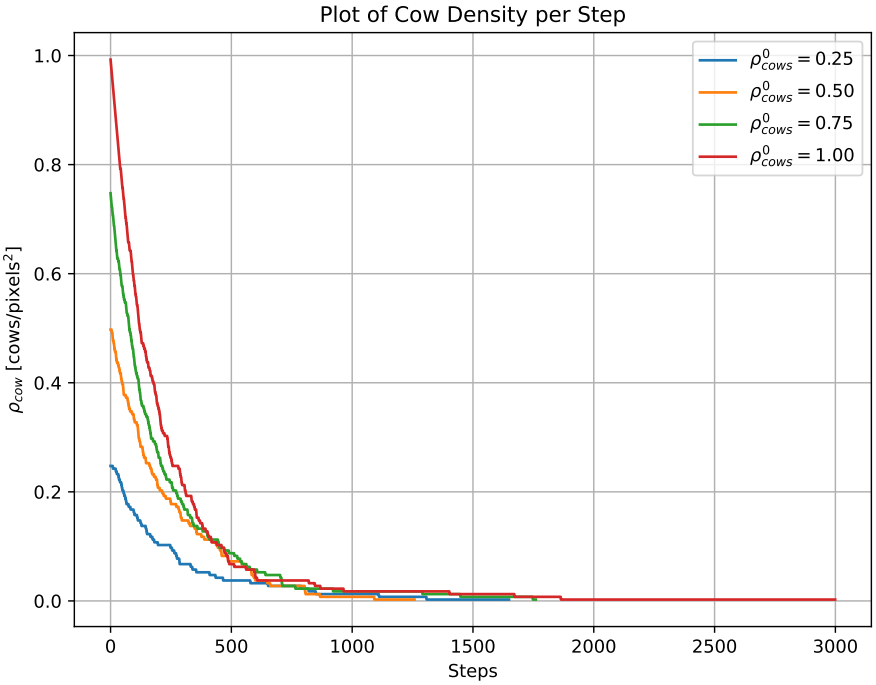


Figure 2: This plot shows the decrease of the cow density for each step of the cows make depending on the initial density of cows ρ^0_{cow}. This plot utilized a grid of dimensions 200 pixels $\times$ 200 pixels.

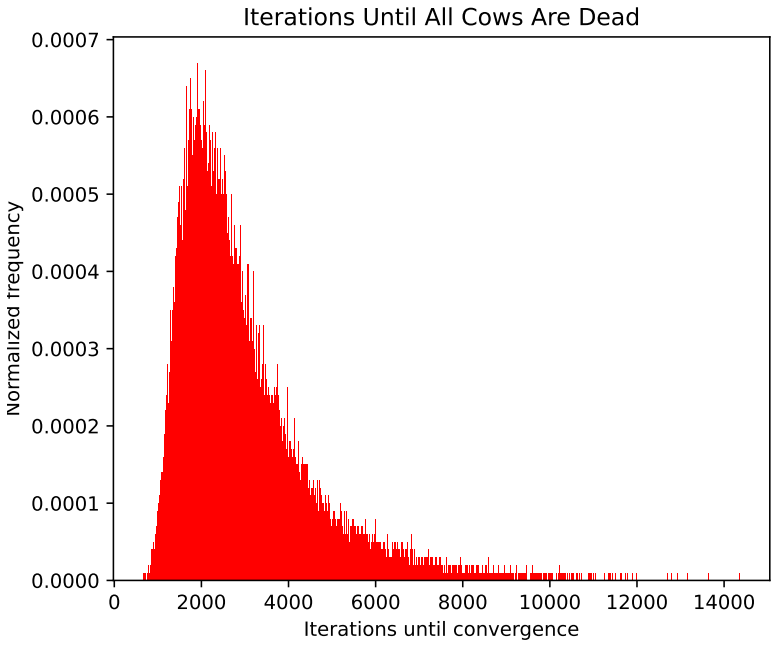
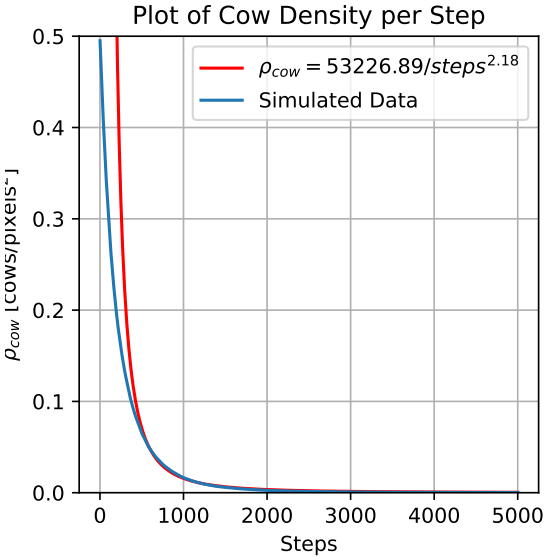
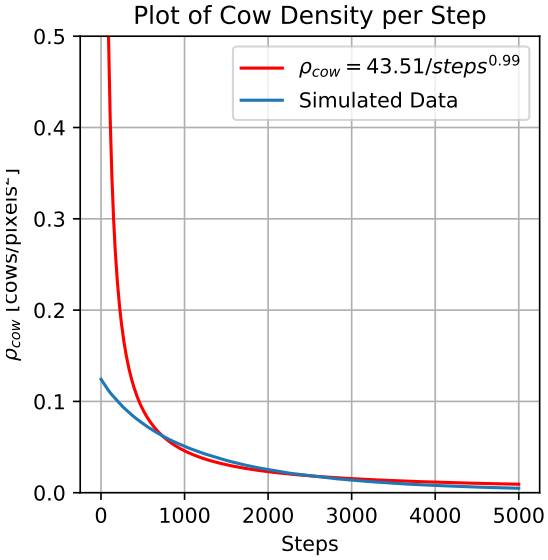


Figure 3: This plot shows the distribution of the number of iterations it took until all cows have died, meaning $\rho_{\text{cow}} = 0$. This plot utilized 100 000 samples and a grid of dimensions 200 pixels $\times$ 200 pixels.



(a) This plot shows the decrease of the cow density with a grid of dimensions 200 pixels × 200 pixels, with a curve fit from Equation 1. This yielded an exponent of $n = 2.18$.



(b) This plot shows the decrease of the cow density with a grid of dimensions 400 pixels × 400 pixels, with a curve fit from Equation 1. This yielded an exponent of $n = 0.99$

Figure 4: These plots use the mean iteration from 100 iterations and also display a curve fit of the theoretical equation (disregarding the first 10% of the values in steps).

3.2 Cows on a Mine Field

Some snapshots from the video simulation can be seen in Figure 5. Viewing the images, one can see that the cow indeed walks as it should, obeying all the rules stipulated above. The iteration in Figure 5 showed that it took nineteen cows, with a total of 192 steps, for the cows to cross the field. However, the number of cows it took to cross the field, $N_{\text{dead cows}}$, varied depending on the simulation. To find the mean of $N_{\text{dead cows}}$, one can observe Figure 6. The bar chart showed a distribution that closely resembled a normal Gaussian distribution. Thus, to find the mean number of cows it took to cross the field of a certain depth, one can calculate the mean of a Gaussian fit on the distribution. This yielded a mean $\mu = \langle N_{\text{dead cows}} \rangle = 24$ dead cows for a field depth of 10 pixels. The simulation in Figure 5 showed $N_{\text{dead cows}} = 22$ dead cows, which is reasonable result since the variance is $\sigma^2 = 36[\text{dead cows}]^2$. If the field was twice as deep, with a depth of 20 pixels, then the mean was found to be $\mu = \langle N_{\text{dead cows}} \rangle = 75$ dead cows, as seen in Figure 6b.

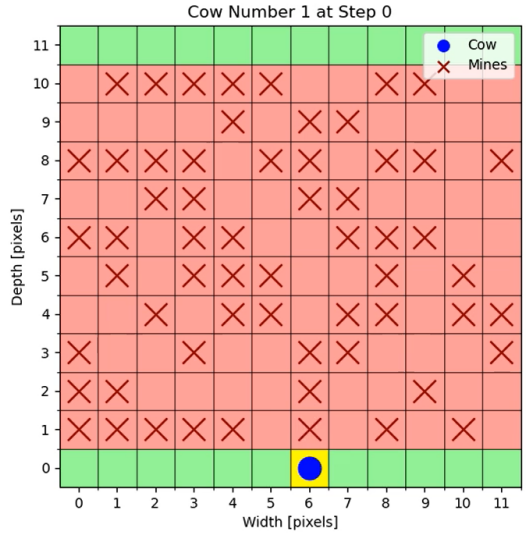
Using this technique of finding the mean for a certain mine density, ρ_{mines}, Figure 7 was created. This figure shows that for a specific field depth (with a width of two additional pixels), a correlation between the mine density, ρ_{mines}, and the mean number of dead cows, $\langle N_{\text{dead cows}} \rangle$, was obtained. For the simulation shown in Figure 5 with a depth of 10 pixels, the correlation was found to be

$$N_{\text{dead cows}} = 55\rho_{\text{mines}} - 3. \tag{2}$$

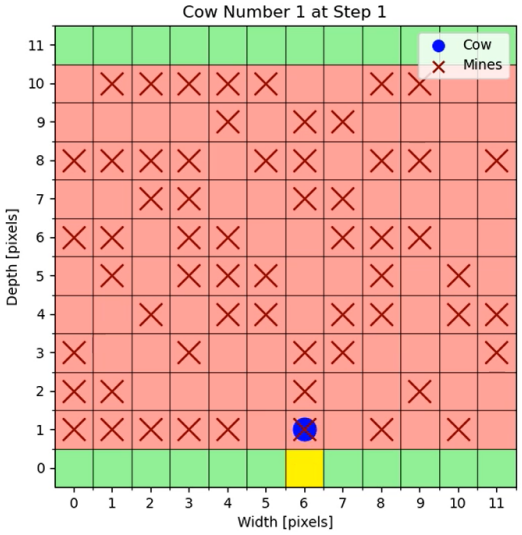
It is important to note that this equation is only defined for $\rho_{\text{mines}} \in [0, 1]\text{mines/pixel}^2$. However, since we cannot have a negative $\langle N_{\text{dead cows}} \rangle$, the interval must be defined as $\rho_{\text{mines}} \in [3/55, 1]\text{mines/pixel}^2$. The reason for this adjustment is that the linear model deviates from the actual correlation as $\rho_{\text{mines}} \longrightarrow 0$. For the simulation where the depth was 20 pixels (and the width was 22 pixels), the linear fit yielded:

$$N_{\text{dead cows}} = 161\rho_{\text{mines}} - 8, \tag{3}$$

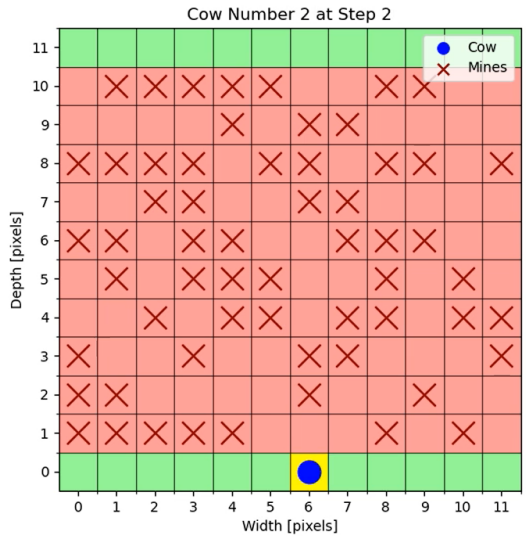
where, again, the interval is defined as $\rho_{\text{mines}} \in [8/161, 1]\text{mines/pixel}^2$. If one wants to find a model that works for low mine densities, a separate low-density function should be fitted instead.



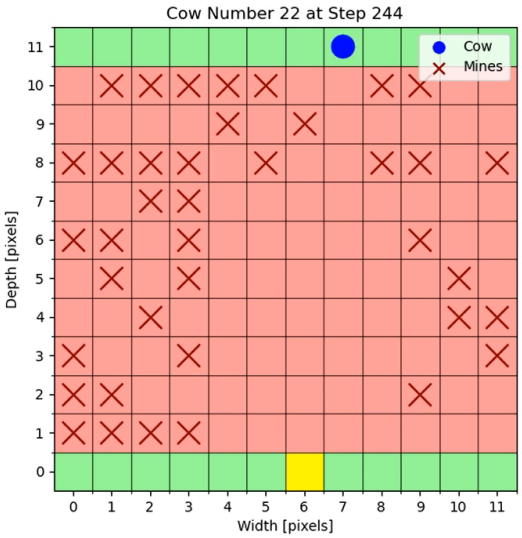
(a) The initial position of the cow.



(b) The cow steps on a mine.

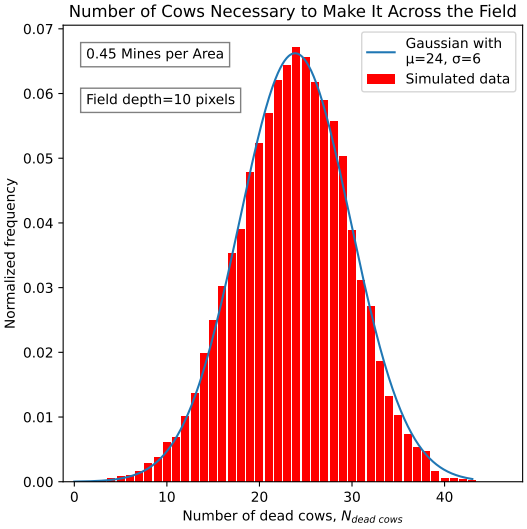


(c) A new cow is introduced on the initial position.

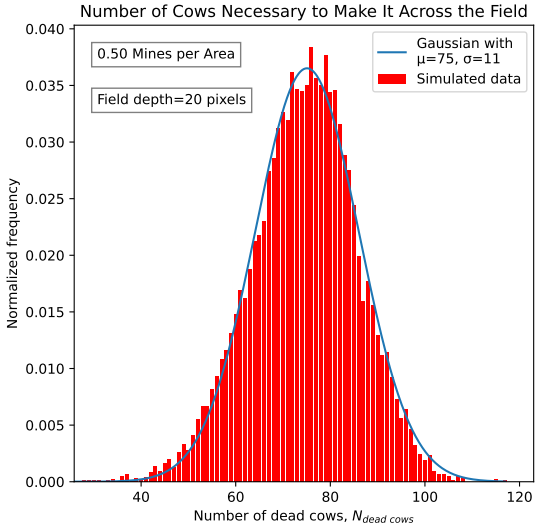


(d) The 22:th cow made it to the other side.

Figure 5: This figure shows a four images from the video obtained. The green area symbolizes the region without any mines, the red are symbolizes the region with mines that the cows want to cross and the yellow area symbolizes the starting position. This iteration was for a field of width 12 pixels, depth of 10 pixels and 60 mines. This gave a mine density of $\rho_{\text{mines}} = 0.5\text{mines/pixels}^2$.

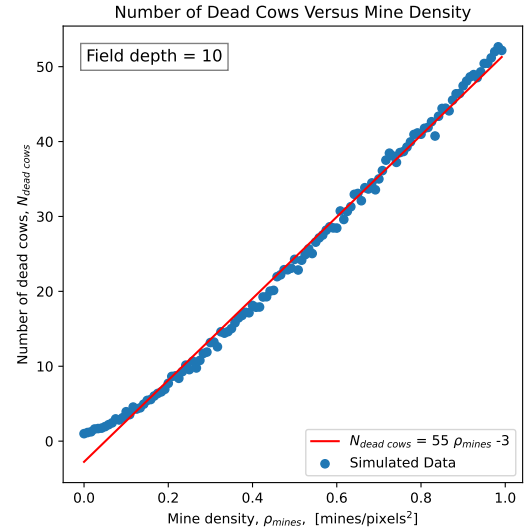


(a) This plot shows the distribution when the minefield had a width of 12 pixels and a depth of 10 pixels. The field contained 54 mines, resulting in a density of 0.45mines/pixels^2. The Gaussian fit yielded a mean of $\mu = 24$ dead cows and a variance of $\sigma^2 = 36$[dead cows]^2.

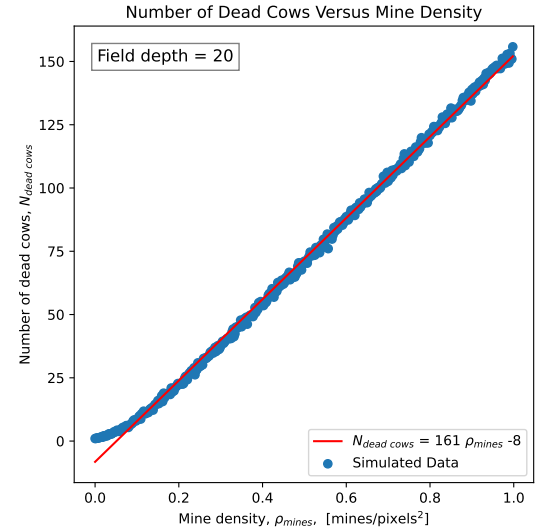


(b) This plot shows the distribution when the minefield had a width of 22 pixels and a depth of 20 pixels. The field contained 229 mines, resulting in a density of 0.50mines/pixels^2. The Gaussian fit yielded a mean of $\mu = 75$ dead cows and a variance of $\sigma^2 = 121$[dead cows]^2.

Figure 6: These figures show the normal Gaussian distribution of the number of cows necessary to make it across the mine field $N_{\text{dead cows}}$. Since the plot showed a normal distribution a Gaussian was fitted to the data. This yielded a variance and more importantly a mean. Both plots were done with a sample size of 10 000.



(a) This graph shows the correlation when the field has a width of 12 pixels and a depth of 10 pixels. The linear fit yielded a function of the form $N_{\text{dead cows}} = 55\rho_{\text{Mines}} - 3$.



(b) This graph shows the correlation when the field has a width of 22 pixels and a depth of 20 pixels. The linear fit yielded a function of the form $N_{\text{dead cows}} = 161\rho_{\text{Mines}} - 8$.

Figure 7: These graphs show the correlation between mine density and the number of dead cows required to cross the field for a specific field depth. The data was fitted with a linear function to represent the correlation. As can be observed, the linear function fits the data well, except for a slight deviation when the mine density is low.

4 Conclusion

4.1 Vicious Walking Cows

In conclusion, the algorithm successfully simulated the behaviour of vicious walking cows. The numeric simulation coincided with the theoretical Equation 1, with an exponent $n \approx 1$, when the system was $160\,000\text{pixels}^2$. It is notable that this type of model is also applicable to many other scenarios, such as molecular interactions, soap bubble dispersion, and particle-particle interactions. Interestingly, all of these scenarios are expected to produce a distribution resembling a Gaussian with an exponential tail. If more time had been available for this project, further analysis on the nature of this distribution would have been interesting.

4.2 Cows on a Minefield

The results from the simulation revealed a linear relationship between mine density and the number of cows required to cross the field for a given field depth, with the results being normally Gaussian distributed. This implies that a higher mine density leads to a greater number of cow fatalities. The linear fit provided a reliable method for predicting the number of dead cows needed to cross the field at any given depth. Although this was a simplified example, this type of modelling can be useful for predicting outcomes in similar systems involving random obstacles and dependencies.

References

[1] Victor Olariu. Computational physics fytn03 - module c lecture 3: Continuous transformation method, 2024. Lecture notes on numerical methods and stochastic processes.

A The python code

A.1 Vicious walking cows

```
# -*- coding: utf-8 -*-
"""
Created on Thu Oct 17 14:35:56 2024

@author: Fredrik Bergelv
"""

import numpy as np
import matplotlib.pyplot as plt
import random as rd
from matplotlib.animation import FuncAnimation, FFMpegWriter
from collections import Counter
import time
from scipy.optimize import curve_fit



def initialize_grid(size, number):
    "This function initializes the cow field"

    grid = np.zeros((size, size))  # We create a grid
    grid_flatt = grid.flatten()    # We flatten the grid to make it simpler

    cow_indices = rd.sample(range(len(grid_flatt)), number+1)  # Randomize positions of the cows

    for i, index in enumerate(cow_indices):
        grid_flatt[index] = 1  # We give all the cows a unique odd number

    moving_cow = rd.choice(cow_indices)

    return grid_flatt, moving_cow  # We return the flattened grid and one random cow


def move_func(moving_cow, grid_flatt, size):
    "This function makes the cow move in a random way"

    paths = []
    # Handle boundaries to prevent out-of-bounds moves
    if not moving_cow % size == 0:          # Left boundary
        paths.append(1)                     # Chance of moving left
    if not moving_cow % size == size - 1:   # Right boundary
        paths.append(2)                     # Chance of moving right
    if moving_cow >= size:                  # Bottom boundary
        paths.append(3)                     # Chance of moving down
    if moving_cow < len(grid_flatt) - size: # Top boundary
        paths.append(4)                     # Chance of moving up

    walk_direction = rd.choice(paths)  # Randomize direction

    # Move left
    if walk_direction == 1:
```

```
        grid_flatt[moving_cow - 1] += grid_flatt[moving_cow]
        grid_flatt[moving_cow] = 0
        moving_cow -= 1  # Move the index to the new position

    # Move right
    if walk_direction == 2:
        grid_flatt[moving_cow + 1] += grid_flatt[moving_cow]
        grid_flatt[moving_cow] = 0
        moving_cow += 1  # Move the index to the new position

    # Move down
    if walk_direction == 3:
        grid_flatt[moving_cow - size] += grid_flatt[moving_cow]
        grid_flatt[moving_cow] = 0
        moving_cow -= size  # Move the index to the new position

    # Move up
    if walk_direction == 4:
        grid_flatt[moving_cow + size] += grid_flatt[moving_cow]
        grid_flatt[moving_cow] = 0
        moving_cow += size  # Move the index to the new position

    return moving_cow, grid_flatt


def Imshow_animate(size, number, steps, style='video'):
    """
    This function runs the cow simulation and generates the animated imshow plot.
    """
    start_time = time.time()

    # Initialize everything
    grid_flatt, moving_cow = initialize_grid(size, number-1)
    frames = np.zeros((steps, size, size))  # We want to store all the grids

    # Track cow positions over time
    for i in range(steps):
        # Make a cow step
        moving_cow, grid_flatt = move_func(moving_cow, grid_flatt, size)

        if grid_flatt[moving_cow] % 2 == 0:  # if vlaue is 2: kill
            grid_flatt[moving_cow] = 0   # Kill cow
            if np.all(grid_flatt == 0):  # Check if all cows are gone
                print(f"Step {i}: All cows are dead, stopping simulation.")
                steps = i+1  # Update the number of frames
                break

        moving_cow = rd.choice(np.where(grid_flatt != 0)[0])  # Find new random cow

        grid = grid_flatt.reshape((size, size))
        frames[i] = grid  # Update the frame
        if i ==steps-1:
            print(f"Step {i}: The cows did not go extinct!")

    # Create the imshow animation
    fig, ax = plt.subplots(figsize=(8, 8))
    im = ax.imshow(frames[0], cmap='jet')  # Use the first frame as the initial image
```

```
    # Set axis properties (similar to the other code)
    ax.set_xticks(np.arange(0, size, 1))
    ax.set_yticks(np.arange(0, size, 1))
    ax.set_xticks(np.arange(0.5, size + 0.5, 1), minor=True)
    ax.set_yticks(np.arange(0.5, size + 0.5), minor=True)

    ax.set_xlabel("X-axis")
    ax.set_ylabel("Y-axis")
    ax.set_title("Killer Cow Simulation at step 0")

    def update(frame):
        if 100*frame % steps==0: # We print each time we do one percent
            print(f'{round(100*frame/steps)}%')

        im.set_array(frames[frame])  # Update the image with the current frame
        ax.set_title(f"Killer Cow Simulation at step {frame}")  # Update the title
        return [im]

    # Create and run the animation using the pre-generated frames array
    animation = FuncAnimation(fig, update, frames=steps, blit=True, repeat=False)

    # Optionally, save the animation as a GIF or video
    if style=='video':
        animation.save(f'Imshow(size={size},number={number},steps={steps}).mp4',
                       writer=FFMpegWriter(fps=60))
    if style=='gif':
        animation.save(f'Imshow(size={size},number={number},steps={steps}).gif',
                       writer='pillow', fps=60)

    end_time = time.time()
    print(f"Elapsed time: {round(end_time - start_time)} seconds")


def Sactter_animate(size, number, steps, style='video'):
    """
    This function runs the cow scatter simulation and generates the animated scatter plot.
    """

    start_time = time.time()

    # Initialize everything
    grid_flatt, moving_cow = initialize_grid(size, number-1)
    grid = grid_flatt.reshape((size, size))

    cow_positions = []  # Store cow positions for each step

    # Track cow positions over time
    for i in range(steps):
        # MAke a cow step
        moving_cow, grid_flatt = move_func(moving_cow, grid_flatt, size)

        if grid_flatt[moving_cow] % 2 == 0:  # If value is 2: kill
            grid_flatt[moving_cow] = 0  # Kill cows
            if np.all(grid_flatt == 0):  # Check if all cows are gone
                print(f"Step {i}: All cows are dead, stopping simulation.")
                steps = i+1
                break
```

```
        moving_cow = rd.choice(np.where(grid_flatt != 0)[0])  # Find new random cow

        # Extract cow positions (x,y), by finding coordinates in flatt and thus normal grid
        current_cow_positions = np.array(np.unravel_index(np.where(grid_flatt != 0)[0],
                                                          grid.shape)).T
        cow_positions.append(current_cow_positions)
        if i ==steps-1:
            print(f"Step {i}: The cows did not go extinct!")

    last_frame = np.array([])  # Add an empty frame at the end
    cow_positions.append(last_frame)

    # Create the scatter plot animation
    fig, ax = plt.subplots(figsize=(6, 6))
    ax.set_xlim(-0.5, size-1)
    ax.set_ylim(-0.5, size-1)
    ax.set_xlabel("Width")
    ax.set_ylabel("Depth")
    ax.set_title("Cow Simulation at Step 0")
    ax.set_xticks(np.arange(0, size, 1))
    ax.set_yticks(np.arange(0, size, 1))
    ax.set_xticks(np.arange(0.5, size+0.5, 1), minor=True)
    ax.set_yticks(np.arange(0.5, size+0.5), minor=True)
    ax.grid(True, which='minor', color='black', linewidth=0.5)

    # Function to update the scatter plot
    def update(frame):

        if 100*frame % steps==0: # We print each time we do one percent
            print(f'{round(100*frame/steps)}%')

        ax.clear()  # Clear the previous plot
        ax.set_xlim(-0.5, size-1)
        ax.set_ylim(-0.5, size-1)
        ax.set_xlabel("Width [pixels]")
        ax.set_ylabel("Depth [pixels]")
        ax.set_title(f"Cow Simulation at Step {frame:.0f}")
        ax.set_xticks(np.arange(0, size, 1))
        ax.set_yticks(np.arange(0, size, 1))
        ax.set_xticks(np.arange(0.5, size+0.5, 1), minor=True)
        ax.set_yticks(np.arange(0.5, size+0.5), minor=True)
        ax.grid(True, which='minor', color='black', linewidth=0.5)

        # Get cow positions for the current frame if we have cows
        if cow_positions[frame].size > 0:  # If there are still cows
            x, y = cow_positions[frame][:, 1], cow_positions[frame][:, 0]
            ax.scatter(x, y, c='red', label='Cows', s=3.8e4/size**2, marker='o')
            ax.legend(loc='upper right', markerscale=1)

    # Create and run the animation using the pre-generated cow positions
    animation = FuncAnimation(fig, update, frames=steps, blit=False, repeat=False)

    # Optionally, save the animation as a GIF or video
    if style=='video':
        animation.save(f'Sactter(size={size},number={number},steps={steps}).mp4',
                       writer=FFMpegWriter(fps=60))
    if style=='gif':
        animation.save(f'Sactter(size={size},number={number},steps={steps}).gif',
```

```
                        writer='pillow', fps=60)

    end_time = time.time()
    print(f"Elapsed time: {round(end_time - start_time)} seconds")


def Statistics_plot(size, number, steps, sample, save=False):
    """ This function displays how many iterations we need to kill all cows"""
    def counter(size=size, number=number, steps=steps, sample=sample):
        """
        This function runs the cow simulation for many different times.
        """

        iteration_list = []  # We want to store all the values
        for k in range(sample):
            if 100*k % sample==0: # We print each time we do one percent
                print(f'{round(100*k/sample)}%')

            # Initialize everything
            grid_flatt, moving_cow = initialize_grid(size, number-1)

            for i in range(steps):
                # Make a cow step
                moving_cow, grid_flatt = move_func(moving_cow, grid_flatt, size)

                if grid_flatt[moving_cow] % 2 == 0:  # If value is 2: Kill
                    grid_flatt[moving_cow] = 0  # Kill cow
                    if np.all(grid_flatt == 0):  # Check if all cows are gone
                        iteration_list.append(i)  # Update the number of frames
                        break

                moving_cow = rd.choice(np.where(grid_flatt != 0)[0])  # Find new random cow

        return iteration_list

    # Count all the iterations we gathered
    iteration_list = (counter(size=size, number=number, steps=steps, sample=sample))
    iteration_list_count = Counter(iteration_list)

    # Sort the iteration_list by unique values
    unique_values = sorted(iteration_list_count.keys())
    frequencies = np.array([iteration_list_count[val] for val in unique_values])
    frequencies = frequencies / np.sum(frequencies)

    # Create the bar chart
    plt.figure(figsize=(6, 5))
    plt.bar(unique_values, frequencies, color='red')

    # Add labels and title
    plt.xlabel('Iterations until convergence')
    plt.ylabel('Normalized frequency')
    plt.title('Iterations Until All Cows Are Dead')

    # Display the chart
    plt.show()

    # Optionally save
    if save == True :
```

```
        plt.savefig(f'Statistics(depth={size},number={number},sample={sample}).pdf')


def Convergence_plot(size, number, steps, save=False):

    sample = 1000  # Number of iterations for averaging

    density_list = np.zeros((steps, sample))  # Store density values for all iterations

    for k in range(sample):
        # Initialize the grid and cows
        grid_flatt, moving_cow = initialize_grid(size, number-1)

        for i in range(steps):
            # Make a cow step
            moving_cow, grid_flatt = move_func(moving_cow, grid_flatt, size)

            if grid_flatt[moving_cow] % 2 == 0:  # If value is 2: kill cow
                grid_flatt[moving_cow] = 0  # Kill cow
                if np.all(grid_flatt == 0):  # Check if all cows are gone
                    break

            # Find a new random cow
            moving_cow = rd.choice(np.where(grid_flatt != 0)[0])

            # Calculate the density of cows
            density = (len(np.where(grid_flatt != 0)[0]) - 1) / size**2
            density_list[i, k] = density  # Store density for this iteration

    # Calculate the mean density over all iterations
    mean_density = np.mean(density_list, axis=1)

    # Define the fitting function: A / x^n
    def f(x, A, n):
        return A / (x**n)

    # Prepare the data for fitting
    x = np.arange(1, len(mean_density) + 1)  # Steps from 1 to number of steps
    half_index = len(x) // 10  # Find from where we want to fit the data
    last_half_x = x[half_index:]
    last_half_density = mean_density[half_index:]  # Extract the density values

    # Perform curve fitting on the last half
    fit, _ = curve_fit(f, last_half_x, last_half_density)

    # Generate values for the fit line
    x_val = np.linspace(0, max(last_half_x), 1000)

    # Plot the results
    plt.figure(figsize=(4, 4))
    plt.plot(x_val, f(x_val, *fit), c='r',
             label=rf'$\rho_{{cow}} = {fit[0]:.2f}/steps^{{{fit[1]:.2f}}}$')
    plt.ylim(0,0.5)
    plt.plot(x, mean_density, label='Simulated Data')
    plt.grid(True)
    plt.xlabel('Steps')
    plt.ylabel(r'$\rho_{{cow}}$ [cows/pixels$^2$]')
    plt.title('Plot of Cow Density per Step')
```

```
    plt.legend()
    plt.show()

    # Save the plot if needed
    if save:
        plt.savefig(f'Convergence(size={size},number={number},steps={steps}).pdf')


def Many_convergence_plot(size, number_list, steps, save=False):
    """ This shows the convergence for different densities"""
    # Plot the result in the same plot
    plt.figure(figsize=(8, 6))
    plt.grid(True)
    plt.xlabel('Steps')
    plt.ylabel(r'$\rho_{{cow}}$ [cows/pixels$^2$]')
    plt.title('Plot of Cow Density per Step')
    for number in number_list:
        grid_flatt, moving_cow = initialize_grid(size, number-1)

        density_list = []  # We want to store all the values

        for i in range(steps):
            # Make a cow step
            moving_cow, grid_flatt = move_func(moving_cow, grid_flatt, size)

            if grid_flatt[moving_cow] % 2 == 0:  # If value is 2: kill
                grid_flatt[moving_cow] = 0  # Kill cow
                if np.all(grid_flatt == 0):  # Check if all cows are gone
                    break

            moving_cow = rd.choice(np.where(grid_flatt != 0)[0])  # Find new cow

            density = (len(np.where(grid_flatt != 0)[0])-1)/size**2
            density_list.append(density) # Append the result
        # Plot the result
        plt.plot(list(range(len(density_list))), density_list,
                 label=rf'$\rho_{{cows}}^{{0}}=${number/(size**2):.2f}')

    plt.legend()
    plt.show()

    if save == True :
        plt.savefig(f'Convergence2(size={size},number={number_list},steps={steps}).pdf')



#Sactter_animate(size=40, number=200, steps=5000, style='video')

#Convergence_plot(size=20, number=200, steps=5000, save=True)

#Statistics_plot(size=30, number=200, steps=30000, sample=100000, save=True)

#Many_convergence_plot(size=20, number_list=[100, 200, 300, 400], steps=3000, save=True)
```

A.2 Cows on a mine field

```
# -*- coding: utf-8 -*-
"""
Created on Thu Oct 17 14:35:56 2024

@author: Fredrik Bergelv
"""

import numpy as np
import matplotlib.pyplot as plt
import random as rd
from matplotlib.animation import FuncAnimation, FFMpegWriter
from collections import Counter
import time
from scipy.optimize import curve_fit


def initialize_grid(size, number):
    """Initialize the minefield grid with no mines on the t.op and bottom row"""
    grid = np.zeros((size, size)) # Create an empty grid with area size*size
    mine_flatt = grid.flatten()   # Flatten the grid for easier indexing

    # Total cells excluding the first and last rows
    total_cells = (size - 2) * size

    # Ensure we don't request more mines than available cells
    if number > total_cells:
        raise ValueError("Number of mines exceeds available cells in the grid.")

    # Randomize positions of the mines excluding the first and last rows
    available_indices = list(range(size * 1, size * (size - 1)))  # Exclude first and last rows
    mine_indices = rd.sample(available_indices, number)  # Randomly sample mine positions

    for index in mine_indices:
        mine_flatt[index] = 1  # Place mines in the selected positions

    return mine_flatt

def move_func(moving_cow, grid_flatt, size):
    "This function makes the cow move one step in a random direction"

    paths = []
    # Handle boundaries to prevent out-of-bounds moves

    if not moving_cow % size == 0:          # Left boundary
        paths.append(1)                     # As chance of moving left
    if not moving_cow % size == size - 1:   # Right boundary
        paths.append(2)                     # Ad chance of moving right
    if moving_cow >= size:                  # Bottom boundary
        paths.append(3)                     # Ad chance of moving down
    if moving_cow < len(grid_flatt) - size: # Top boundary
        paths.append(4)
        paths.append(4)                     # The chance of moving up should be 50%
        paths.append(4)
        paths.append(4)
```

```
    walk_direction = rd.choice(paths)  # Randomize direction

    # Move left
    if walk_direction == 1:
        grid_flatt[moving_cow - 1] += grid_flatt[moving_cow]
        grid_flatt[moving_cow] = 0
        moving_cow -= 1  # Move the index to the new position

    # Move right
    if walk_direction == 2:
        grid_flatt[moving_cow + 1] += grid_flatt[moving_cow]
        grid_flatt[moving_cow] = 0
        moving_cow += 1  # Move the index to the new position

    # Move down
    if walk_direction == 3:
        grid_flatt[moving_cow - size] += grid_flatt[moving_cow]
        grid_flatt[moving_cow] = 0
        moving_cow -= size  # Move the index to the new position

    # Move up
    if walk_direction == 4:
        grid_flatt[moving_cow + size] += grid_flatt[moving_cow]
        grid_flatt[moving_cow] = 0
        moving_cow += size  # Move the index to the new position

    return moving_cow, grid_flatt # Return the new moving cow and the new grid

def Sactter_animate(size, number, style='video'):

    """This function shows that the cow deaths fit a Gaussian"""


    start_time = time.time()

    # Initialize everything
    grid = np.zeros((size, size))
    mine_flatt = initialize_grid(size, number)
    grid_flatt = initialize_grid(size, 0) # We start with no cow

    cow_positions  = []  # Store cow positions for each step
    mine_positions = []  # Store mine positions for each step
    cow_counter    = []  # Store which cow iteration we are on

    frames = 0 # Count how many frames we want to display
    cow = 1 # Count which cow we are on

    done = False
    while done==False: # Iterate until one cow makes it across the field
        frames += 1
        moving_cow = int(size/2) # The starting poition of the cow
        grid_flatt[moving_cow] = 1

        # Update all the lists where we store our values on the form (x,y)
        current_cow_positions = np.array(np.unravel_index(np.where(grid_flatt != 0)[0],
                                                           grid.shape)).T
        cow_positions.append(current_cow_positions)
        cow_counter.append(cow)
```

```
        current_mine_positions = np.array(np.unravel_index(np.where(mine_flatt != 0)[0],
                                                           grid.shape)).T
        mine_positions.append(current_mine_positions)

        for i in range(1000): # Loop through the movment of the cow
            frames += 1

            moving_cow, grid_flatt = move_func(moving_cow, grid_flatt, size)  # Make a cow step

            # Update all the lists where we store our values on the form (x,y)
            current_cow_positions = np.array(np.unravel_index(np.where(grid_flatt != 0)[0],
                                                              grid.shape)).T
            cow_positions.append(current_cow_positions)
            cow_counter.append(cow)
            current_mine_positions = np.array(np.unravel_index(np.where(mine_flatt != 0)[0],
                                                               grid.shape)).T
            mine_positions.append(current_mine_positions)

            # Since all cows are 1, if position is 2: kill
            if (grid_flatt[moving_cow]+mine_flatt[moving_cow]) == 2:
                grid_flatt[moving_cow] = 0  # Kill cows
                mine_flatt[moving_cow] = 0  # Remove the mine
                cow += 1

                print(f"Step {i}: The cow stepped on a mine.")
                break

            # Check if we are on the other side
            if moving_cow > len(grid_flatt) - size:
                print(f"Step {i}: The cow made it across the mine field.")
                done = True # Stop the while-loop
                break


    # Create the scatter plot animation
    fig, ax = plt.subplots(figsize=(6, 6))
    ax.set_xlim(-0.5, size-1)
    ax.set_ylim(-0.5, size-1)
    ax.set_xlabel(r"Width $\left[\text{pixels}\right]$")
    ax.set_ylabel(r"Depth $\left[\text{pixels}\right]$")
    ax.set_title("Cow Number 1 at Step 0")
    ax.set_xticks(np.arange(0, size, 1))
    ax.set_yticks(np.arange(0, size, 1))
    ax.set_xticks(np.arange(0.5, size+0.5, 1), minor=True)
    ax.set_yticks(np.arange(0.5, size+0.5), minor=True)
    ax.grid(True, which='minor', color='black', linewidth=0.5)

    # Function to update the scatter plot
    def update(frame):
        if 100*frame % frames==0: # We print each time we do one percent
            print(f'{round(100*frame/frames)}%')

        ax.clear()  # Clear the previous plot
        ax.set_xlim(-0.5, size-1)
        ax.set_ylim(-0.5, size-1)
        ax.set_xlabel("Width [pixels]")
        ax.set_ylabel("Depth [pixels]")
        ax.set_title(f"Cow Number {cow_counter[frame]:.0f} at Step {frame:.0f}")
```

```
        ax.set_xticks(np.arange(0, size, 1))
        ax.set_yticks(np.arange(0, size, 1))
        ax.set_xticks(np.arange(0.5, size+0.5, 1), minor=True)
        ax.set_yticks(np.arange(0.5, size+0.5), minor=True)
        ax.grid(True, which='minor', color='black', linewidth=0.5)

        # Color the grid to make it easier to view
        grid_colors = np.ones((size, size, 3))  # Initialize grid with white (all ones)

        # Light green for top and bottom rows
        grid_colors[0, :, :] = [0.6, 1.0, 0.6]    # Green for bottom rows
        grid_colors[-1, :, :] = [0.6, 1.0, 0.6]   # Green for top rows
        grid_colors[1:-1, :, :] = [1.0, 0.6, 0.6] # Red for middle rows
        grid_colors[0, int(size/2), :] = [1.0, 1.0, 0.0] # Color of starting position

        # Plot grid as background
        ax.imshow(grid_colors, extent=[-0.5, size-0.5, -0.5, size-0.5], origin='lower')

        # Get cow positions for the current frame if we have cows
        if cow_positions[frame].size > 0:  # If there are still cows
            # Plot x, y positions for cow
            x, y = cow_positions[frame][:, 1], cow_positions[frame][:, 0]
            ax.scatter(x, y, c='blue', label='Cow', s=3.8e4/size**2, marker='o')
            # Plot x, y positions for mines
            x_mine, y_mine = mine_positions[frame][:, 1], mine_positions[frame][:, 0]
            ax.scatter(x_mine, y_mine, c='darkred', label='Mines', s=3.8e4/size**2, marker='x')

        ax.legend(loc='upper right', markerscale=1/2)


    # Create and run the animation using the pre-generated cow positions
    animation = FuncAnimation(fig, update, frames=frames, blit=False, repeat=False)

    # Optionally, save the animation as a GIF or video
    if style=='video':
        animation.save(f'Mine_field(size={size},number={number},steps={frames}).mp4',
                       writer=FFMpegWriter(fps=2))
    if style=='gif':
        animation.save(f'Mine_field(size={size},number={number},steps={frames}).gif',
                       writer='pillow', fps=10)

    end_time = time.time()
    print(f"Elapsed time: {round(end_time - start_time)} seconds")

def counter(size, number, sample):
    """ This function runs the cow simulation for many different times."""

    iteration_list = []  # We want to store all the values for how long it took to get across

    for k in range(sample): # We print each time we do one percent

        # Initialize everything
        grid_flatt = initialize_grid(size, 0)
        mine_flatt = initialize_grid(size, number)

        cow_counter = []

        frames = 0
```

```
        cow = 1
        # Track cow positions over time
        done = False
        while done==False:
            frames += 1
            moving_cow = int(size/2)
            grid_flatt[moving_cow] = 1

            cow_counter.append(cow)

            for i in range(1000):
                frames += 1
                # Make a cow step
                moving_cow, grid_flatt = move_func(moving_cow, grid_flatt, size)

                # append the cow position
                cow_counter.append(cow)
                # If the cow steps on a mine kill everything
                if (grid_flatt[moving_cow] + mine_flatt[moving_cow]) == 2:
                    grid_flatt[moving_cow] = 0  # Kill cows
                    mine_flatt[moving_cow] = 0  # Remove the mine
                    cow += 1
                    break
                # Check if the cow made it across the field
                if moving_cow > len(grid_flatt) - size:
                    done = True
                    iteration_list.append(cow) # Add the number of iterations
                    break

    return iteration_list

def Statistics_plot(size, number, sample, save=False):
    """ This function shows how many iterations it takes to make it across"""

    iteration_list = counter(size=size, number=number, sample=sample)
    iteration_list_count = Counter(iteration_list) # Count all the iterations

    # Sort the iteration_list by unique values
    unique_values = np.array(sorted(iteration_list_count.keys()))
    frequencies = np.array([iteration_list_count[val] for val in unique_values])
    frequencies = frequencies/ np.sum(frequencies)

    # Create the bar chart
    plt.figure(figsize=(6, 6))
    plt.bar(unique_values, frequencies, color='red', label='Simulated data')

    plt.xlabel(r'Number of dead cows, $N_{{dead \ cows}}$')
    plt.ylabel('Normalized frequency')
    plt.title('Number of Cows Necessary to Make It Across the Field')
    plt.text(0.05, 0.95, f'{number/(size*(size-1)):.2f} Mines per Area',
             transform=plt.gca().transAxes,
         fontsize=10, verticalalignment='top', horizontalalignment='left',
         bbox=dict(facecolor='white', alpha=0.5))
    plt.text(0.05, 0.85, f'Field depth={size-2} pixels',
             transform=plt.gca().transAxes,
         fontsize=10, verticalalignment='top', horizontalalignment='left',
         bbox=dict(facecolor='white', alpha=0.5))
```

```
    # Calculate the mean
    numerator = sum(value*frequency for value, frequency in zip(unique_values, frequencies))
    denominator = sum(frequencies)
    mean = numerator / denominator
    mean = mean
    print(f'The mean is {mean:.2f} iterations.')

    # Do a gaussian fit
    def gaussian(x, mu, sigma):
        coefficient = 1 / (np.sqrt(2 * np.pi * sigma ** 2))
        exponent = np.exp(-((x - mu) ** 2) / (2 * sigma ** 2))
        pdf = coefficient * exponent
        return pdf

    initial_mu = np.mean(unique_values)  # mean of your unique_values
    initial_sigma = np.std(unique_values)  # standard deviation of your unique_values

    # Fit the data
    fit = curve_fit(gaussian, unique_values, frequencies, p0=[initial_mu, initial_sigma])
    x_val = np.linspace(0,max(unique_values),1000)
    plt.plot(x_val, gaussian(x_val, fit[0][0], fit[0][1]),
             label = f'Gaussian with\n mu={fit[0][0]:.0f}, sigma={np.abs(fit[0][1]):.0f}')
    plt.legend(loc='upper right')

    plt.xlim(mean - 2 * initial_sigma, mean + 2 * initial_sigma)  # Adjust the range as needed

    plt.show()

    if save == True :
        plt.savefig(f'Statistics_mine_field(depth={size-2},number={number},sample={sample}).pdf')

def Optimizing_plot(size, sample, save=False):
    """ This function shows the """

    number_cows  = [] # Store number of cows
    mine_density = [] # Store mine density per depth (same as per area)

    for mine_number in range(size*(size-2)):
        if 100*mine_number % (size*(size-2))==0: # We print each time we do one percent
            print(f'{round(100*mine_number/(size*(size-2)))}%')

        # Count the result
        iteration_list = counter(size=size, number=mine_number, sample=sample)
        iteration_list_count = Counter(iteration_list)

        # Extract the result
        unique_values = np.array(sorted(iteration_list_count.keys()))
        frequencies = np.array([iteration_list_count[val] for val in unique_values])

        # calculate the mean
        numerator = sum(value*frequency for value,frequency in zip(unique_values,frequencies))
        denominator = sum(frequencies)
        mean = numerator / denominator
        mean = mean
        number_cows.append(mean)
        mine_density.append(mine_number/(size*(size-2)))

    # Plot the result
```

```
    plt.figure(figsize=(6, 6))
    plt.xlabel(r'Mine density, $\rho_{{mines}}$,  [mines/pixels$^2$]')
    plt.ylabel(r'Number of dead cows, $N_{{dead \ cows}}$')
    plt.title('Number of Dead Cows Versus Mine Density')
    plt.text(0.05, 0.95, f'Field depth = {size-2}', transform=plt.gca().transAxes,
         fontsize=12, verticalalignment='top', horizontalalignment='left',
         bbox=dict(facecolor='white', alpha=0.5))

    plt.scatter(mine_density, number_cows, label='Simulated Data')

    # Do a linear fit of the data
    def f(x, A, B):
        return A*x+B

    fit = curve_fit(f, mine_density, number_cows)

    x_val = np.linspace(0,max(mine_density), 1000)
    plt.plot(x_val, f(x_val, fit[0][0], fit[0][1]), c='r',
             label = rf'$N_{{dead \ cows}}$ = {fit[0][0]:.0f} $\rho_{{mines}}$ {fit[0][1]:.0f}')

    plt.legend(loc='lower right')
    plt.show()

    # Optinally save
    if save == True :
        plt.savefig(f'Optimizing_mine_field(depth={size-2},sample={sample}).pdf')



Sactter_animate(size=12, number=60, style='video')

Statistics_plot(size=22, number=229, sample=10000, save=True)
Statistics_plot(size=12, number=60,  sample=10000, save=True)

Optimizing_plot(size=12, sample=100, save=True)
Optimizing_plot(size=22, sample=100, save=True)
```